Monarch (Team 36) Final Report

Alan Munschy (apm238), Bryan Kim (bjk228), Joseph Kunken (jbk232)

The monarch robot reached the semifinals of the Mechatronics robotic cube collecting competition. In this document we review our design and reflections from the competition.

Design & Strategy

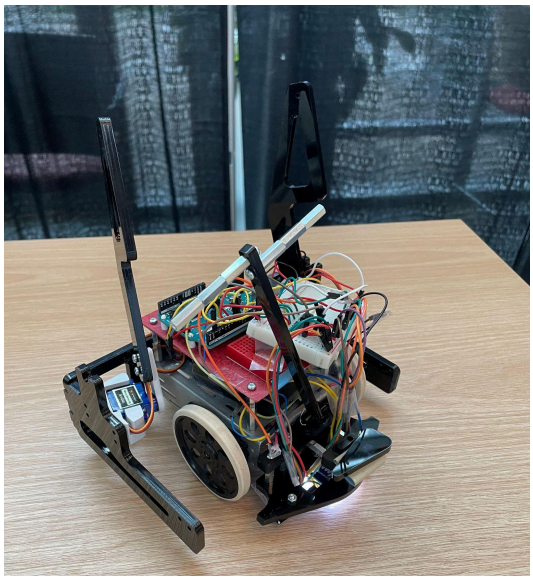


Figure 1.1 Pre-deployed robot

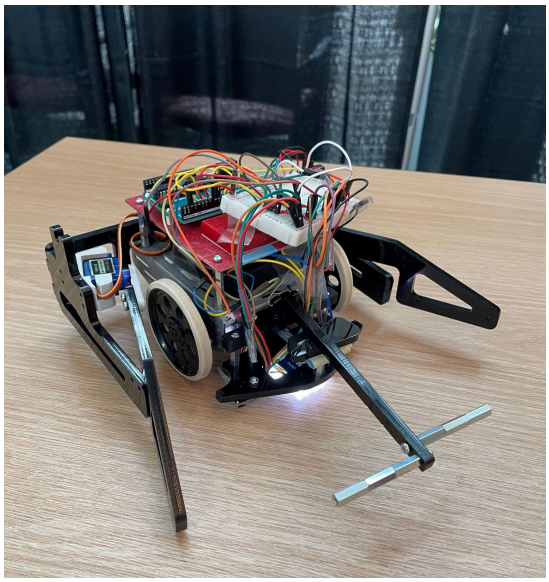


Figure 1.2 Deployed robot

Our robot had a single color sensor and two QTI sensors attached on a custom mount, and utilized two motors for the wheels and two servos for the arms. The arms start pointing upward at a 90 degree angle and once the robot is activated, they come down parallel to the ground. The central arm also starts pointing up and uses inertia when the robot suddenly drives forward and stops, at which point it comes down. Ideally, the two arms serve to maximize the “collection zone” and the central “attack arm” is a combat feature, which either tangles the opponent’s wires or latches onto it and drags it along with ours, which prevents them from collecting any more cubes. We orient the robot at a 45 degree angle from the back edge and it moves forward until it senses the blue-yellow border, turns 135 degrees to move along the border where the cubes are to maximize the amount of cubes collected early on. After that, it goes straight until it senses a block border, and depending on how many QTI sensors sense it, turns and repeats to collect the remaining cubes until time runs out.

Design Process

As motors for the wheels were more or less the same across all teams, we breezed through milestones 1 and 2 without any issues. For milestone 3, we planned to add a custom 3D printed part that connected to the metal chassis and connected to a acrylic laser cut V frame (similar to a cowcatcher on a train) and use one color sensor. However, we ran into issues where our robot could sense the black border and respond accordingly, but it couldn't tell the difference between its starting color and the opposite color, so it would drive across the full length of the board and only turn at the opposite border. We decided to add a QTI sensor for additional data and decided to average its readings with the color sensor. This improved our robot so that it could do Milestone 3 when it started on yellow, but when it started on blue, it would either take a long time to steer away from the black border when it was detected or ignore it entirely. We fixed this by adding a second QTI sensor so that the robot could know what angle it hit a border at (i.e. two QTIs reading black means it hit the border head on and only one meant that it hit the border from the side). This change fixed all the color sensor issues and we passed Milestone 3. The cowcatcher design was effective at diverting all the cubes the robot hit to either side so we passed Milestone 4 without adding the full arms; the back plate (refer to Figure C.9) was enough to do the job.

For the servo arms that extended to collect cubes, there were two designs considered. The first design (see Appendix C.16) rotated the entire arms outward to get a wide cube collecting space. However, this design was awkward because the rules specify that the robot starts in an 8"x8" square and can expand to a 12" diameter cylinder, but only an 8.49"x8.49" square can be inscribed in a cylinder, so having rectangular arms meant there was not a great advantage to dropping arms this way. Having micro servos be the only attachment point for these huge arms was also a concern. If an enemy robot slammed into them, the servos' internal gearing could be damaged. The final design (see Appendix C.1) split the arm into a base area and an extension arm. The base area optimized the starting 8"x8" square space, and the expander area extended arms at a 20 degree angle. This allowed the arms to be wider allowing for a funnel, while staying in the 12" diameter cylinder. The extender arms also rested on the base arm, meaning that the servos did not have to work against gravity, and the servos had a smaller load because they were only carrying an extender instead of the entire arm. This arm design was a major reason for our success because we were able to consistently gather cubes.

Competition Analysis

Our robot performed well during the competition, with a record of 6 wins and 1 tie during the round robin portion, then 2 wins during the main tournament, before losing in the semifinals. Our opening strategy was effective, with our robot starting at a 45 degree angle, then turning to move along the board center. This helped avoid collisions while quickly reaching the center. Our block collection system also worked well, being able to collect many blocks. Our central "attack arm" also was able to disconnect an opponent's wire in one match which appeared to hamper their movement. However the downside of this arm was that it also snagged onto the enemy robot, hampering our own movement and progress.

Some unexpected and weak points in our design and competition performance were our use of rubber bands around our wheels, and the open slots along the length of our arms. When being pushed sideways by other robots, the rubber bands often came undone, reducing traction. Next, in one match an opponent's arms became stuck in the slots along our arms, leading to them levering our arm upwards and losing all of our cubes. Also, a loose color sensor wire went unnoticed between matches, leading to our robot failing to make its initial turn at the board center, reducing cube collection. Finally, our robot frequently only made left turns, leading to cubes overflowing on the right side, while our left side remained relatively empty. This led to losing cubes when hitting the border as they were pushed off the board.

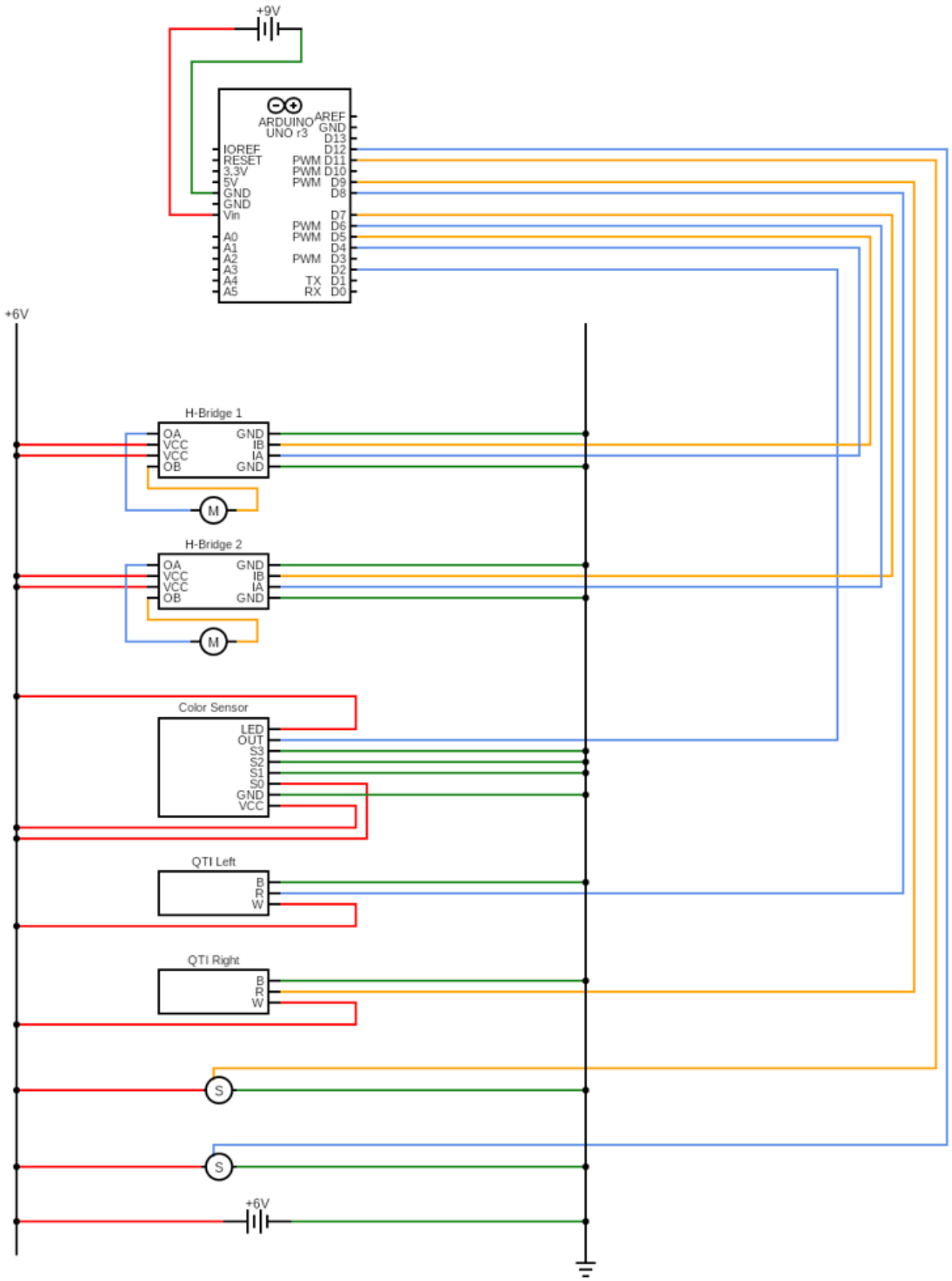
Conclusions

Our robot performed very well on competition day, with us getting 2nd place in the round robin based on win-loss-draw count, and 3rd place in the playoffs and we were pleased with how we designed it. Given a redo, we could have looked into overvolting by wiring the 6 V and 9 V batteries in series rather than using larger wheels, which would have increased the motor's rotation rate while still maintaining control and torque, as the team that beat us (and got 1st in the playoffs) had an extremely fast robot by using that strategy. We believe we could have had a much higher chance of beating them if we used it as well. For students who will compete next year, we would advise to have friendly matches with other teams before the competition to tweak the collection mechanism and to make it as wide as possible to maximize the number of cubes collected. Also, it's essential to have a path/algorithm that balances how many cubes a robot collects and whether or not it will cross paths with the opponent's robot, as if they do, the robot will have to be well designed to win the ensuing shoving match.

Appendix A (Bill of Materials)

	Part	Type	Info	Quantity	Unit Price	Total Price		Total
2	Front Cube Divider	Laser	19.88 in	1	$1.49	$1.49		$39.27
3	Arm Extender	Laser	18.00 in	2	$1.40	$2.80		
4	Arm Base	Laser	26.40 in	2	$1.82	$3.64		
5	Arm Holder	Laser	34.21 in	1	$2.21	$2.21		
6	Attack arm	Laser	12.25 in	1	$1.11	$1.11		
7	Attack mount	Laser	8.34 in	1	$0.92	$0.92		
8	Attack holder	Laser	1.99 in	2	$0.60	$1.20		
9	Nose head mount	PLA	2.49 g	1	$2.00	$2.00		
10	Divider mount	PLA	4.48 g	1	$2.79	$2.79		
11	Servo horn holder	PLA	1.40 g	2	$1.56	$3.12		
12	Servo mount	PLA	5.77 g	2	$3.31	$6.62		
13	Base mount holder	PLA	2.22 g	1	$1.89	$1.89		
14	Positional micro servo	Shop	From lab	1	$3.00	$3.00		
15	12" x 12" Acrylic Sheet	Shop	From RPL	1	$5.00	$5.00		
16								
17	Parts not used							
18	Front Cube Divider	Laser	19.63 in	1	$1.48	$1.48		
19								

Appendix B (Circuit Diagram)



Appendix C (CAD Files & Drawings)

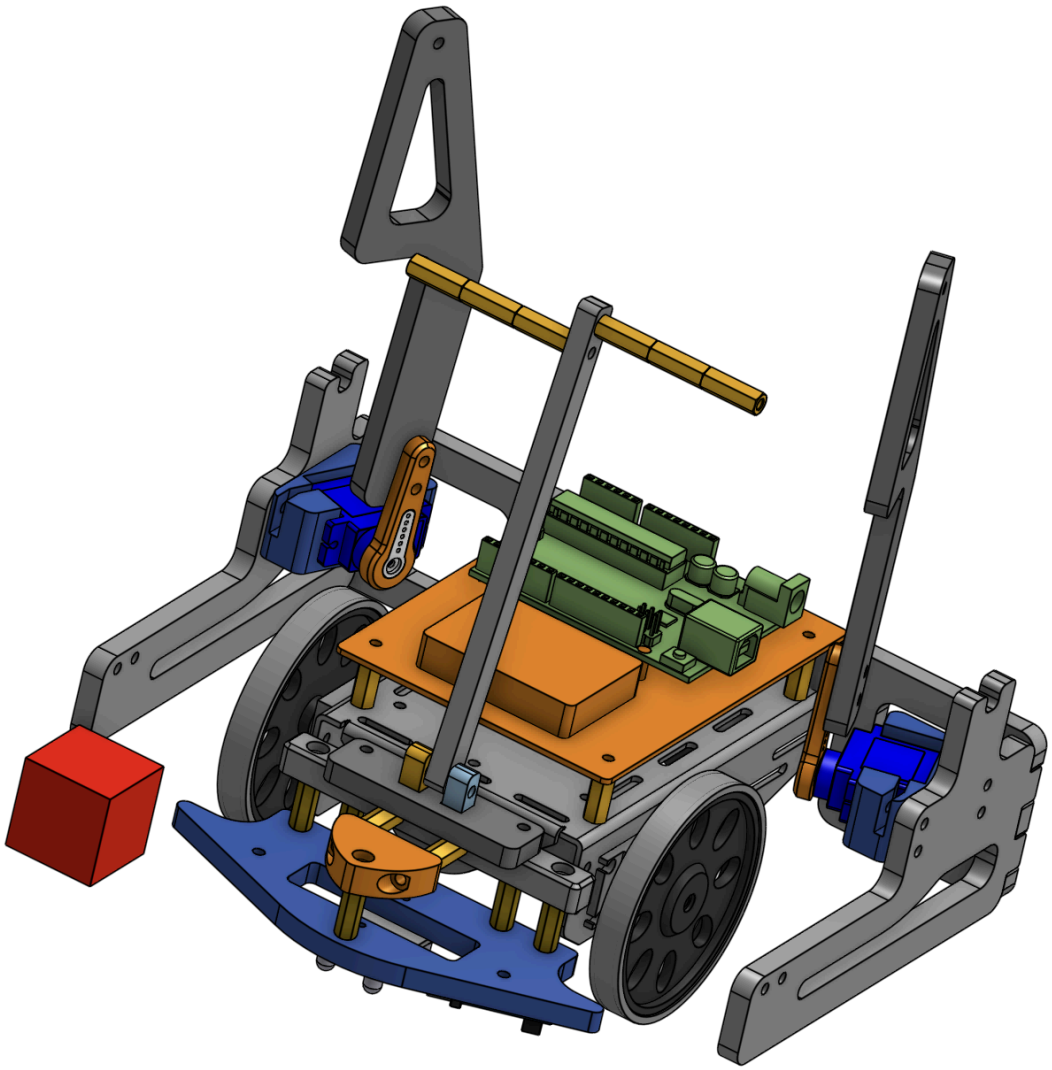


Figure C.1: Isometric view of robot in pre-deployed state to fit 8”x8” square.

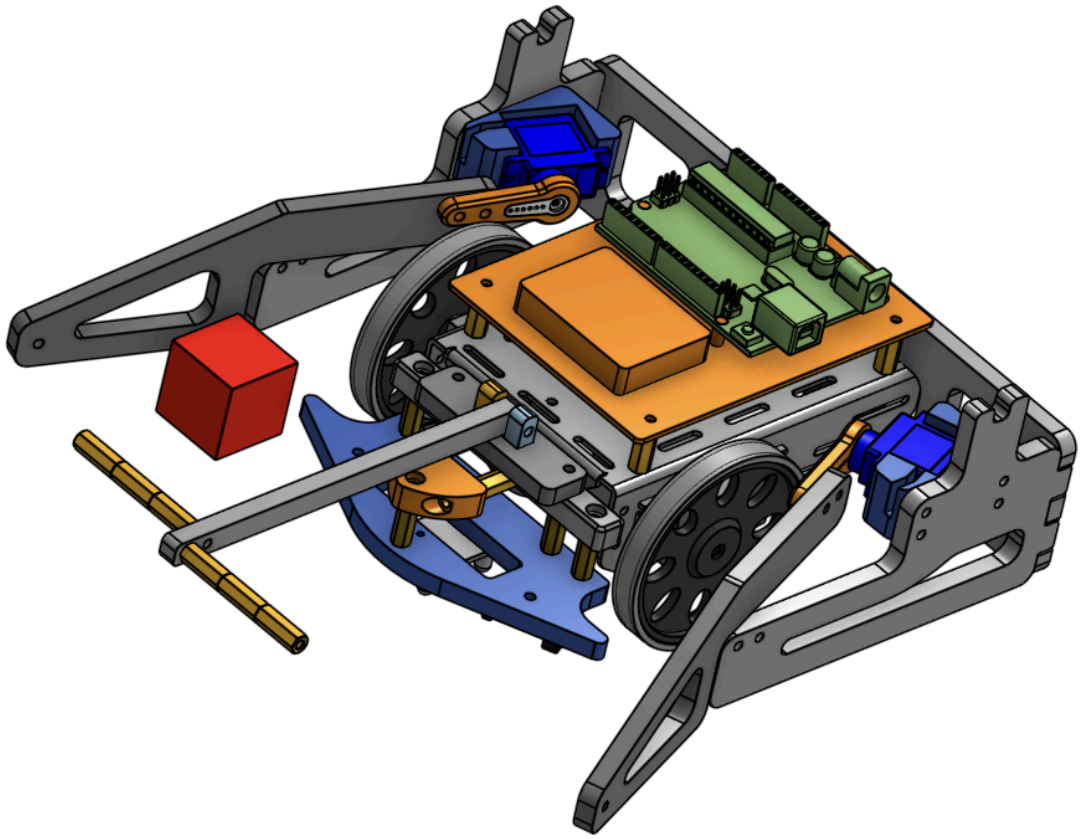


Figure C.2: Isometric view of robot in deployed state (contained in a 12” cylinder).

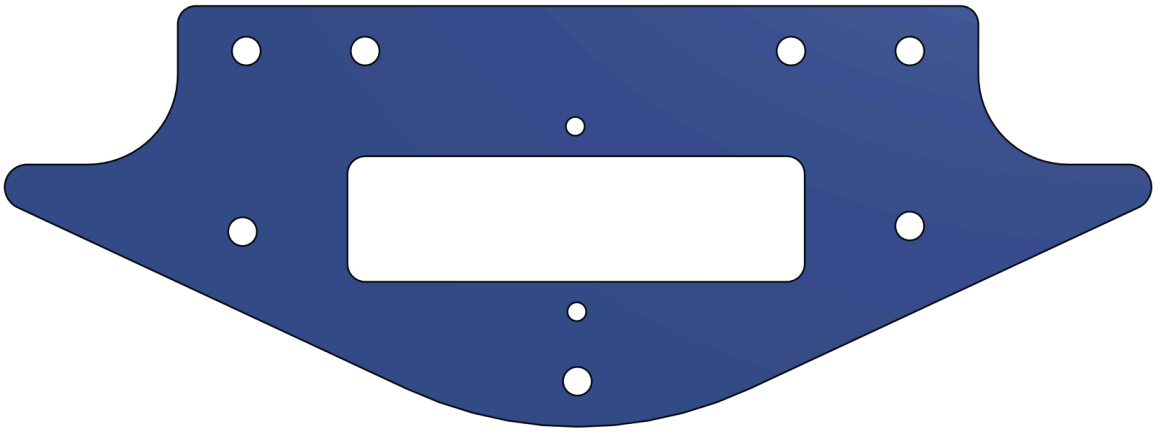


Figure C.3: (Laser Cut) Front cube divider, final version to hold 2 QTI sensors and color sensor. 19.88” cutting length. BOM #2

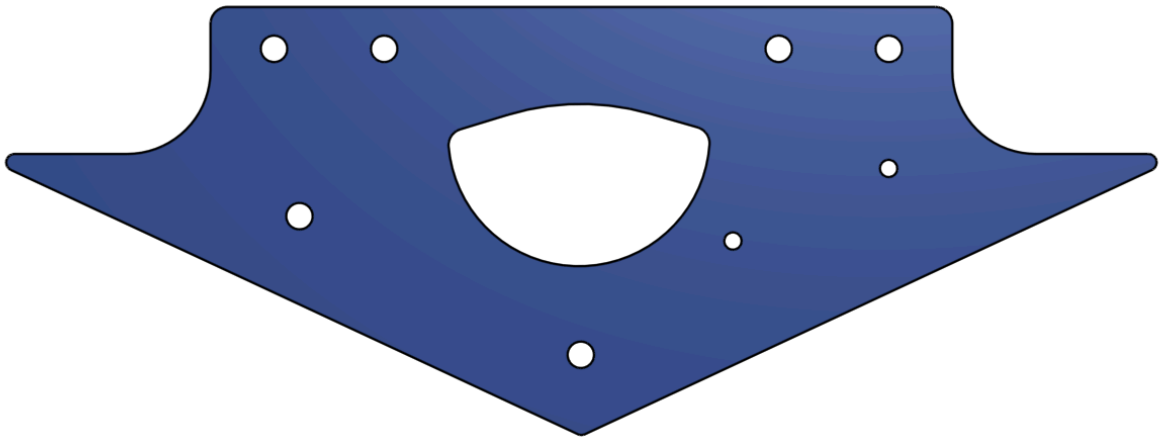


Figure C.4: (Laser Cut) Original Front cube divider, only held 1 QTI sensor and color sensor. Also was longer which prevented blocks from being funneled. 19.63” cutting length. BOM #18

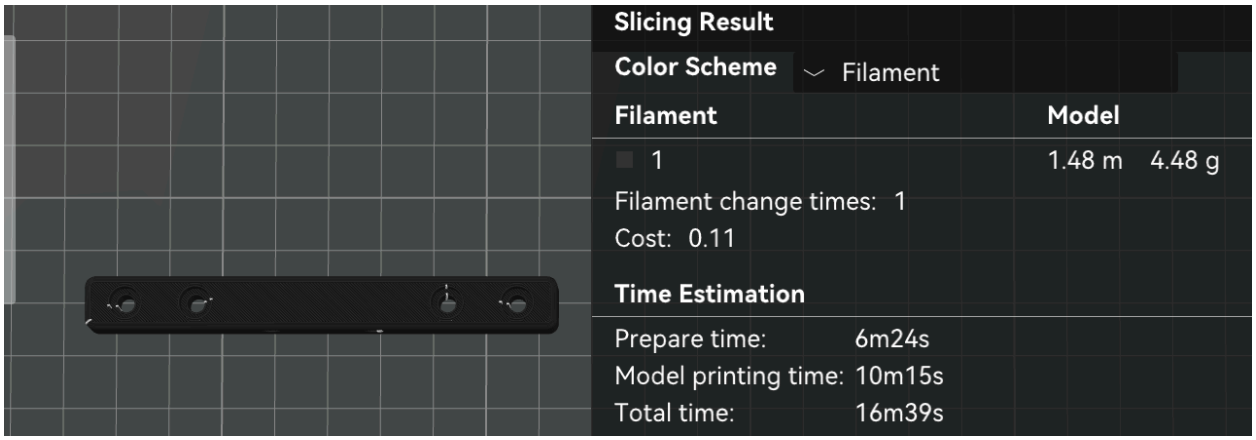


Figure C.5 (PLA) Mount for font cube divider, sliced weight 4.48g. BOM #10

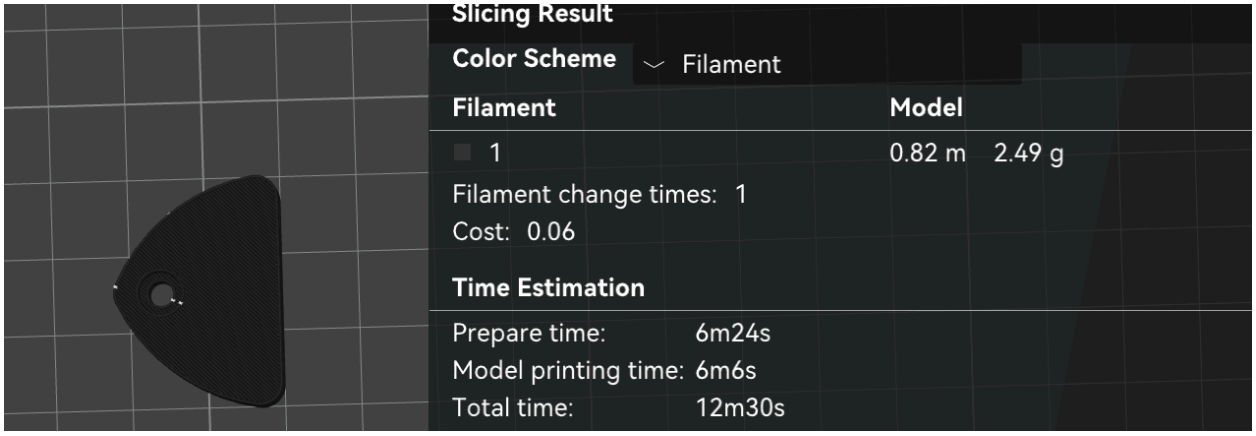


Figure C.6: (PLA) Nose for font cube divider, sliced weight 2.49g. BOM #9

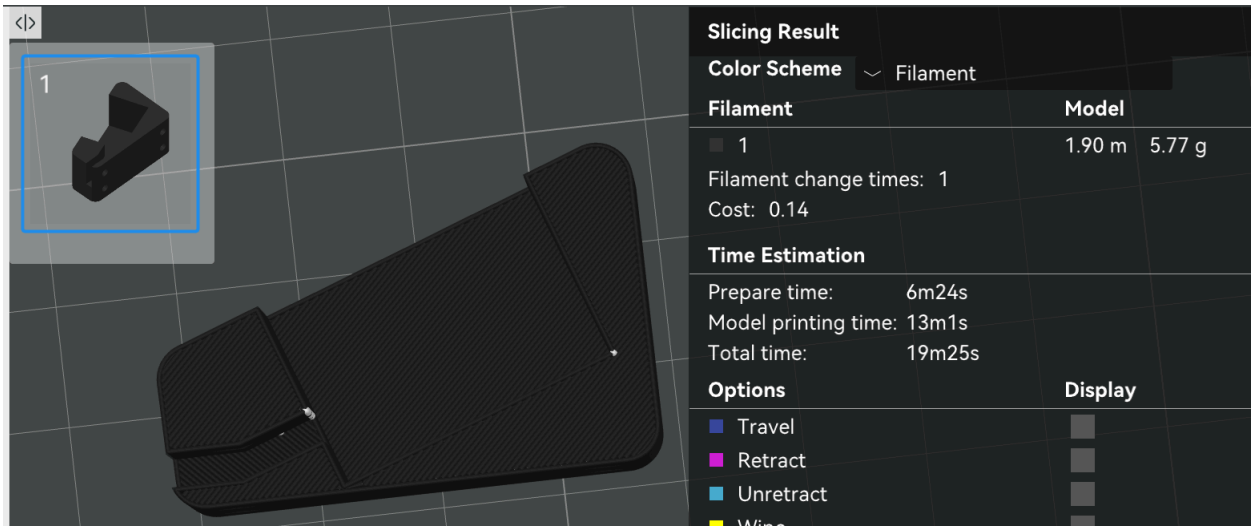


Figure C.7: (PLA) Servo holder, sliced weight 5.77g. BOM #12

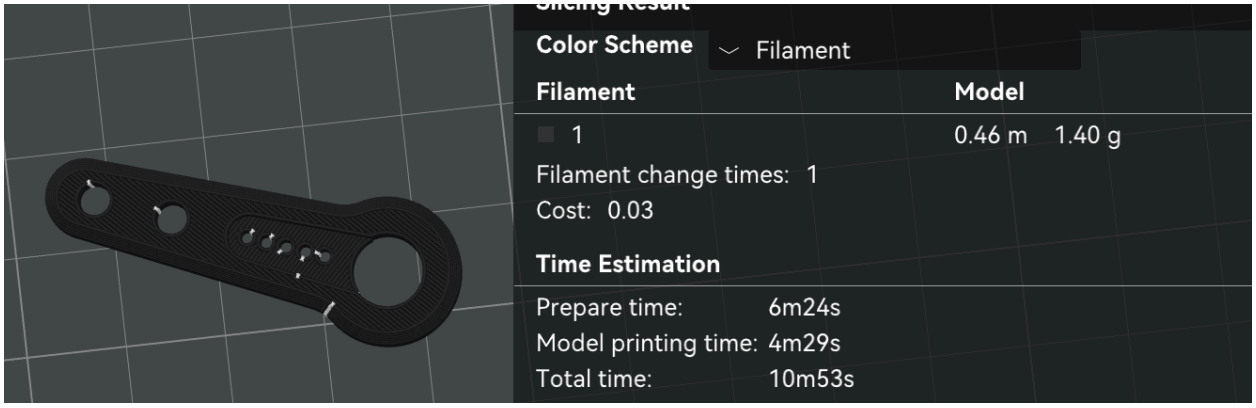


Figure C.8: (PLA) Servo horn holder, sliced weight 1.40g. BOM #11

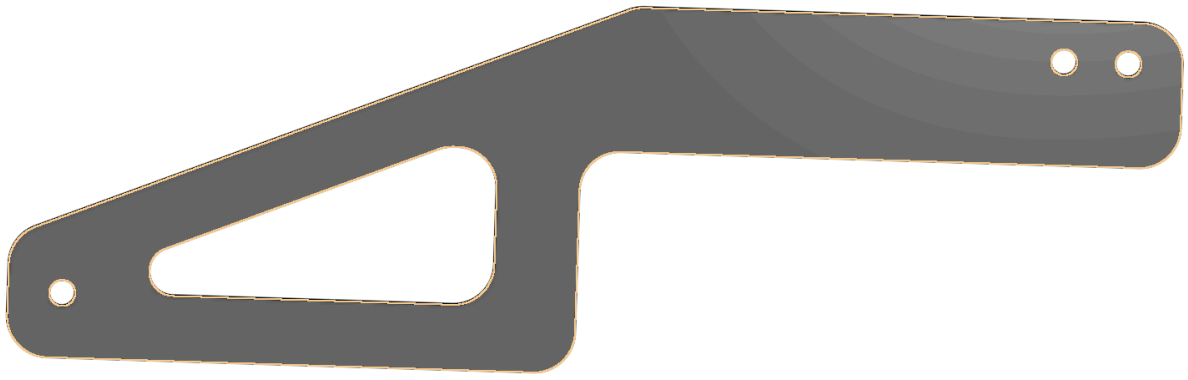


Figure C.9: (Laser Cut) Arm extender for robot. 18” cutting length. BOM #3

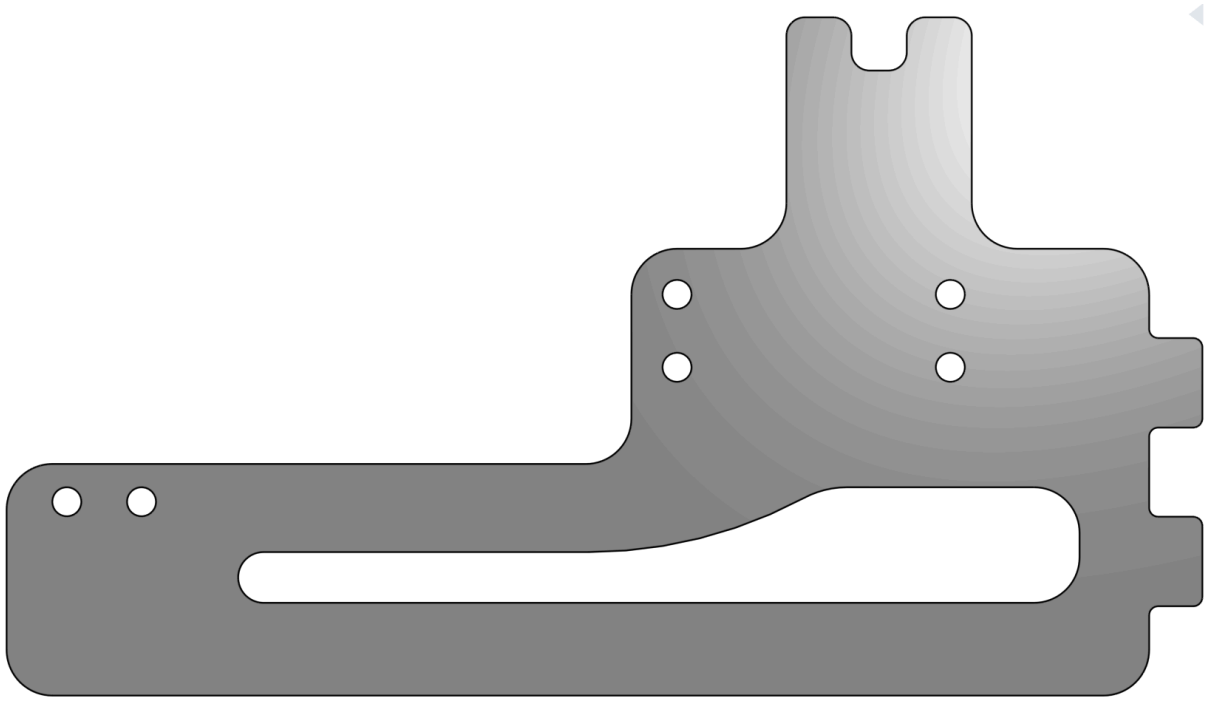


Figure C.10: (Laser Cut) Arm base for robot. 26.40” cutting length. BOM #4

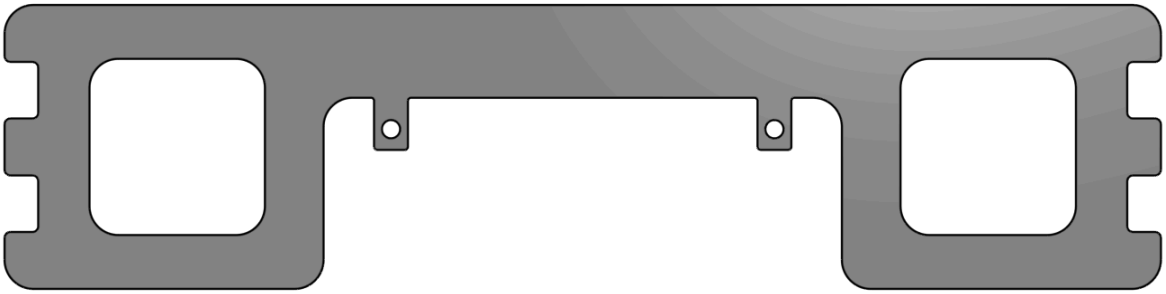


Figure C.11: (Laser Cut) Arm holder for robot. 26.40” cutting length. BOM #5

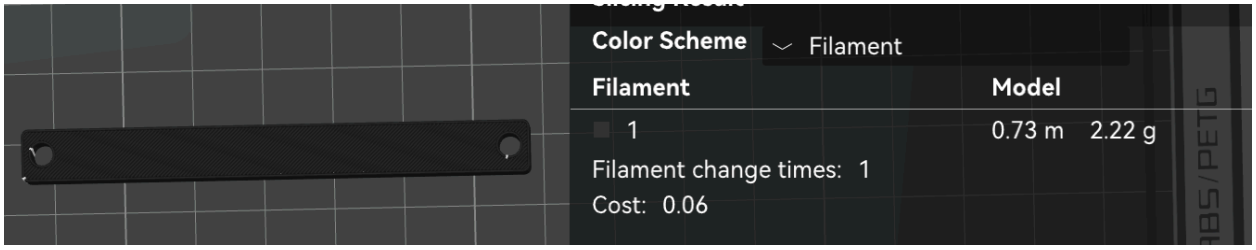


Figure C.12: (PLA) Base mount holder, sliced weight 2.22g. BOM # 13

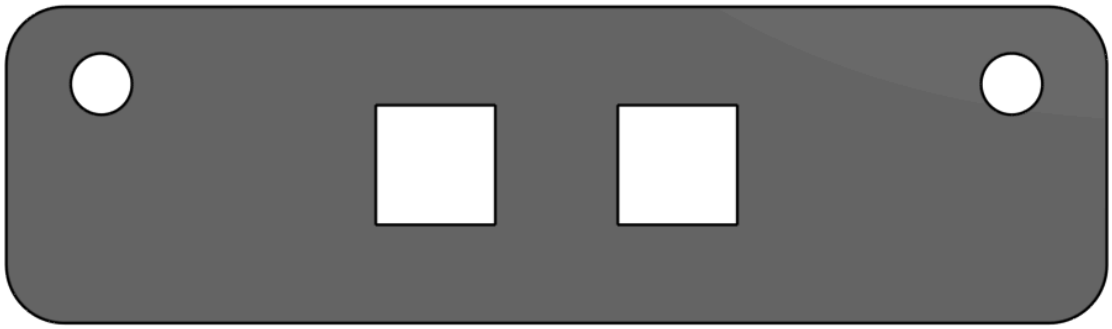


Figure C.13: (Laser Cut) Attack arm mount, 8.34” cutting length. BOM #7

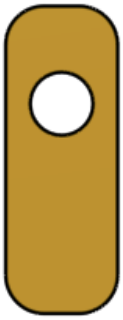


Figure C.14: (Laser Cut) Attack holder, 1.99” cutting length. BOM #8

Figure C.15: (Laser Cut) Attack arm, 12.25” cutting length. BOM #6

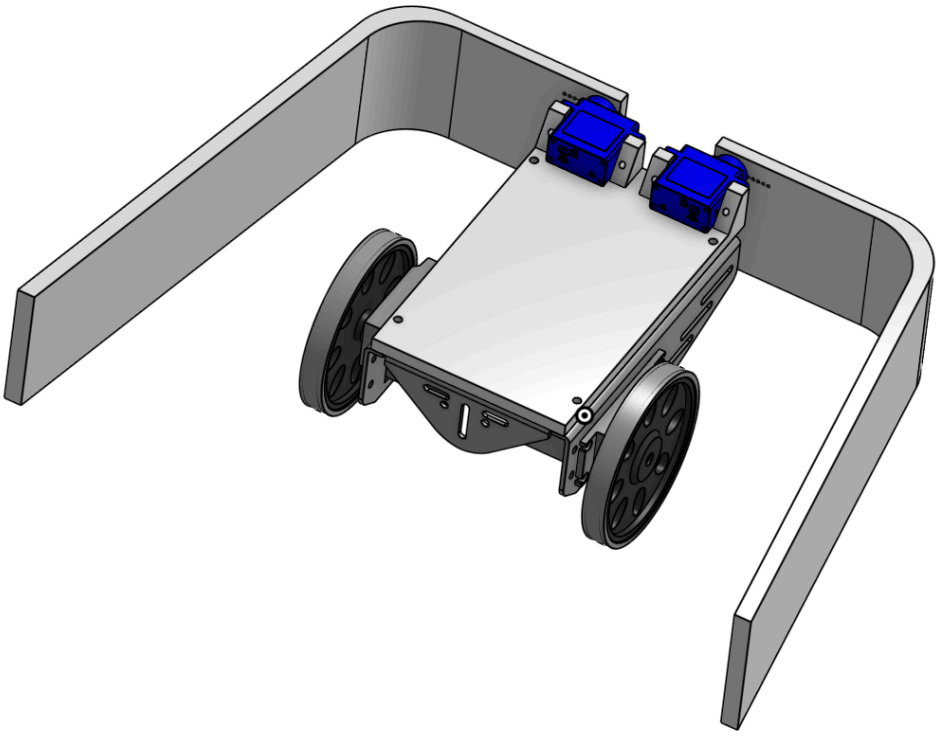
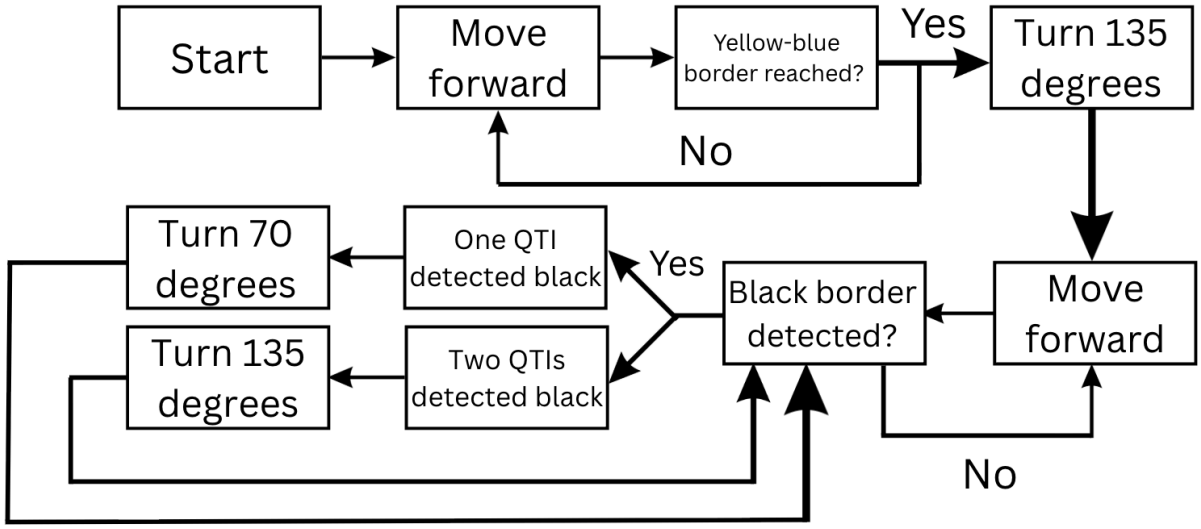


Figure C.16: Original Arm Design

Appendix D (Block Diagram)



Appendix E (Arduino Codes)

```
/*

Left wheel H-bridge:
Input A --> pin 4
Input B --> pin 5
Output A --> motor black wire
Output B --> motor red wire

Right wheel H-bridge:
Input A --> pin 6
Input B --> pin 7
Output A --> motor black wire
Output B --> motor red wire

if either IA and IB are both high or both low, then both OA and OB are low
IA high, IB low --> OA high, OB low
IA low, IB high --> OA low, OB high

Color Sensor Wiring:

LED: 5V power
OUT: Arduino pin 2
S3: ground
S2: ground
S1: ground
S0: 5V power
GND: ground
VCC: 5V power

Left QTI Sensor Wiring:

W: 5V Power
R: Arduino pin 8 (for both input and output)
B: Ground

(black to B, yellow to R)
```

```
Right QTI Sensor Wiring:

W: 5V Power
R: Arduino pin 9 (for both input and output)
B: Ground

(green to B, yellow to R)

Ultrasonic Sensor Wiring (will not be used):

SIG: Arduino pin 10 (for both input and output)

Servo:

Left servo:
brown wire: GND
red wire: power
orange wire: pin 11

right servo:
brown wire: GND
red wire: power
orange wire: pin 12


*/


// Wheel Movement Initializations

#define LEFT_TURN 0
#define RIGHT_TURN 1
#define LEFT_QTI 2
#define RIGHT_QTI 3

//servo
#define LEFT_SERVO_PIN 0b00001000 // Pin 11
#define RIGHT_SERVO_PIN 0b00010000 // Pin 12
```

```
bool forward = false;
bool backward = false;
bool left = false;
bool right = false;
bool stop = false;
bool milestone_2 = false;
bool milestone_3 = false;

float pi = 3.14159;

float wheel_diameter_inches = 2.6;  //measured (inches)
float wheel_radius_inches = wheel_diameter_inches/2.0;
float wheel_radius_feet; //feet


float motor_rpm = 64.32; //measured (rpm)

float motor_angular_speed; //(rad/s)
float speed; //(ft/s)
float wheel_to_wheel_distance = 4.5; //measured between wheel centers
(inches)

float distance_time = 0;
float angle_time = 0;
float arc_distance = 0;
float distance_time_ms = 0;
float angle_time_ms = 0;

float feet = 0;
float meters = 0;
float inches = 0;

bool test = false;
bool repeated_test = false;
bool check_colors_test = false;

float test_angle_multiplier;
float test_distance_multiplier;

int random_angle = 0;
```

```
bool first_color_switch = false;

int previous_turn = LEFT_TURN;

// Color Sensor Initializations

int period;
unsigned int timer;
typedef enum Color{None, Black, Blue, Yellow, Different, Board} The_Color;
The_Color CS_curr_color = None;
The_Color CS_prev_color = None;
bool Yellow_to_Blue = false;
bool Blue_to_Yellow = false;
bool milestone_3_half_achieved = false;
bool initial_color_read = false;
The_Color starting_color = None;
The_Color opposite_color = None;

// QTI Initializations
long duration = 0;
The_Color QTI_left_curr_state = None;
The_Color QTI_right_curr_state = None;

// ultrasonic sensor
volatile uint16_t color_timer_value = 0;

//servo:
uint8_t EEPROM_read(uint16_t address) {
  while(EECR & 0b00000010);
  EEAR = address;
  EECR |= 0b00000001;
  return EEDR;
}

void EEPROM_write(uint16_t address, uint8_t data) {
  while(EECR & 0b00000010);
  EEAR = address;
  EEDR = data;
  EECR |= 0b00000100;
```

```
  EECR |= 0b00000010;
}

void custom_delay_us(uint16_t us) {
  TCNT1 = 0;
  uint32_t target_ticks = (uint32_t)us * 16;

  if (target_ticks > 65530) target_ticks = 65530; // Safety cap

  while (TCNT1 < (uint16_t)target_ticks);
}

//ultrasonic sensor: (not used)
float read_ultrasonic() {
    uint16_t start_ticks, end_ticks;
    uint32_t timeout;

    // --- TRIGGER ---
    DDRB  |=  0b00000100;
    PORTB &= ~0b00000100;
    _delay_us(2);
    PORTB |=  0b00000100;
    _delay_us(5);
    PORTB &= ~0b00000100;

    // --- MEASURE ---
    DDRB &= ~0b00000100;

    uint8_t oldSREG = SREG;
    cli();

    // Safety Timeout 1: Wait for pulse to start (HIGH)
    timeout = 0;
    while (!(PINB & 0b00000100)) {
        if (timeout++ > 50000) { SREG = oldSREG; return -1; }
    }
    start_ticks = TCNT1;

    // Safety Timeout 2: Wait for pulse to end (LOW)
    timeout = 0;
```

```
    while (PINB & 0b00000100) {
        if (timeout++ > 50000) { SREG = oldSREG; return -1; }
    }
    end_ticks = TCNT1;

    SREG = oldSREG;

    uint16_t elapsed = end_ticks - start_ticks;
    return (float)elapsed / 929.0;
}

float ultrasonic_average(){
  float distance1 = read_ultrasonic();
  _delay_ms(5);
  float distance2 = read_ultrasonic();
  _delay_ms(5);
  float distance3 = read_ultrasonic();
  _delay_ms(5);
  float avg_distance = (distance1+distance2+distance3)/3;
  return avg_distance;
}

//interrupt for color sensor timer
ISR(PCINT2_vect){
  if(PIND & 0b00000100){
    TCNT1 = 0;
  }
  else{
    timer = TCNT1;
  }
}

//complete turns that are greater than 45 degrees in multiple steps to
improve accuracy
void large_angle_turn(int direction, int large_angle){
  int number_steps = (large_angle + 89) / 45;
  float turn_size = (float)large_angle/number_steps;

  for(int i = 0; i < number_steps; i++){
    stop_moving_time(30);
```

```
    if(direction == LEFT_TURN){
      turn_left_angle(turn_size);
    }
    else{
      turn_right_angle(turn_size);
    }

  }
  stop_moving_time(100);
}

//obtain color sensor data
int getColor(){
  PCMSK2 = 0b00000100;
  _delay_ms(10);
  PCMSK2 = 0b00000000;
  period = 2*timer*0.0625;
  return period;
}

//define ranges for color sensor period --> color
The_Color CS_Colors(int the_period){
  if(the_period >= 280 && the_period < 500){
    return Black;
  }
  else if(the_period >= 130 && the_period <= 279){
    return Blue;
  }
  else if(the_period >= 10 && the_period <= 129){
    return Yellow;
  }
  else if(the_period == 0){
    return None;
  }
  else{
    return Different;
  }
}

//checking what color is received from color sensor
```

```
The_Color Check_CS(){
  // CS = Color Sensor
  int the_period_1 = getColor();
  _delay_ms(5);
  int the_period_2 = getColor();
  _delay_ms(5);
  int the_period_3 = getColor();
  _delay_ms(5);
  int the_period = (the_period_1 + the_period_2 + the_period_3)/3;
  //take average to account for slight period variations

  The_Color CS_Result = CS_Colors(the_period);
  return CS_Result;
}

void CS_Check_Color_Switch(){ //checking when switching between yellow and
blue sides of the board
  if(CS_prev_color == Yellow && CS_curr_color == Blue){
    Yellow_to_Blue = true;
  }
  else if(CS_prev_color == Blue && CS_curr_color == Yellow){
    Blue_to_Yellow = true;
  }
  else{
    Yellow_to_Blue = false;
    Blue_to_Yellow = false;
  }
}

void Hit_Border_Left_QTI(){ //when hitting border on left side, turn right
  random_angle = 75;
  stop_moving_time(25);
  drive_backward_distance(0.1);
  stop_moving_time(25);
  large_angle_turn(RIGHT_TURN, random_angle);
  stop_moving_time(20);
}
void Hit_Border_Right_QTI(){ //when hitting border on right side, turn
left
  random_angle = 75;
```

```
  stop_moving_time(25);
  drive_backward_distance(0.1);
  stop_moving_time(25);
  large_angle_turn(LEFT_TURN, random_angle);
  stop_moving_time(20);
}

//define ranges for QTI data --> color/border
The_Color QTI_Colors(int QTI_side, long duration){

  if(duration >= 400){
    return Black;
  }
  else if(duration > 0 && duration <= 399){
    return Board;
  }
  else if(duration == 10000){
    return Different;
  }
  else{
  }

}

//get left QTI data
The_Color Check_Left_QTI(){
  duration = 0;
  DDRB = 0b00000001;
  PORTB = 0b00000001;
  _delay_ms(25);
  DDRB = 0b00000000;
  PORTB = 0b00000000;
  while(PINB & 0b00000001 && duration < 10000){
    duration++;
  }

  The_Color QTI_State = QTI_Colors(LEFT_QTI, duration);
  return QTI_State;
}
```

```
//get right QTI data
The_Color Check_Right_QTI(){
  duration = 0;
  DDRB = 0b00000010;
  PORTB = 0b00000010;
  _delay_ms(50);
  DDRB = 0b00000000;
  PORTB = 0b00000000;
  while(PINB & 0b00000010 && duration < 10000){
    duration++;
  }

  The_Color QTI_State = QTI_Colors(RIGHT_QTI, duration);
  return QTI_State;
}

int main(void){
  DDRD = 0b11110000; //outputs: wheel motors (pins 4-7)  | inputs: Color
Sensor (pin 2)
  DDRB = 0b00011000; //inputs: QTI (pins 8, 9)  | outputs: QTI (pins 8,
9), Servos (pin 11, 12) //for now set as input
  PORTD = 0b00000000;
  PORTB = 0b00000000;

  PCICR = 0b00000100;
  PCMSK2 = 0b00000100;

  TCCR1A = 0b00000000;
  TCCR1B = 0b00000001;

  sei();

  // SERVO & EEPROM LOGIC
  uint8_t angle = EEPROM_read(0);
  if (angle > 180) {
    angle = 40;
  } else {
    angle = angle * 2;
    if (angle > 180) angle = 40;
  }
```

```
  EEPROM_write(0, angle);

  // Logic for different servo angles, want servo arms pointed straight
ahead
  uint8_t left_angle = 133;   // 133 is good, higher = higher, lower =
lower
  uint8_t right_angle = 20; // 20 = perfect 30 = lowers the end

  // Calculate pulse widths (1000us to 2000us range)
  uint16_t leftPulse  = 1000 + ((uint32_t)left_angle * 1000 / 180);
  uint16_t rightPulse = 1000 + ((uint32_t)right_angle * 1000 / 180);

  // Send pulses to both servos
  for (int i = 0; i < 50; i++) {
    cli();

    // Pulse Left Servo (Pin 11)
    PORTB |= LEFT_SERVO_PIN;
    custom_delay_us(leftPulse);
    PORTB &= ~LEFT_SERVO_PIN;

    // Pulse Right Servo (Pin 12)
    PORTB |= RIGHT_SERVO_PIN;
    custom_delay_us(rightPulse);
    PORTB &= ~RIGHT_SERVO_PIN;

    sei();
    _delay_ms(18); // Maintain ~20ms period
  }

  //basic speed calculation
  wheel_radius_feet = inches_to_feet(wheel_radius_inches);
  motor_angular_speed = (motor_rpm*2.0*pi)/60.0; //(rad/s)
  speed = wheel_radius_feet*motor_angular_speed;


  //initial_color_read = false;

  //slight calibration for angle turns and distance movement
  test_angle_multiplier = 1.05;
```

```
  test_distance_multiplier = 1.0645;

  while(1){
    //get color sensor and QTI colors
    CS_prev_color = CS_curr_color;
    CS_curr_color = Check_CS();
    QTI_left_curr_state = Check_Left_QTI();
    QTI_right_curr_state = Check_Right_QTI();


    //determine initial color side
    if(initial_color_read == false){
      starting_color = CS_curr_color;
      initial_color_read = true;
    }
    if(starting_color == Yellow){
      opposite_color = Blue;
    }
    else if(starting_color == Blue){
      opposite_color = Yellow;
    }

    //when first hitting opposite color, turn 135 degrees so will be
moving along the middle of the board
    if(CS_curr_color == opposite_color && first_color_switch == false){
      first_color_switch = true;
      stop_moving_time(25);
      large_angle_turn(LEFT_TURN, 135);
      stop_moving_time(25);
      drive_forward();
    }

    //logic for hitting black border
    if(QTI_left_curr_state == Black && QTI_right_curr_state == Black){
//turn 135 degrees to get back to center area when facing head on to the
border
      drive_backward_distance(0.15);
      large_angle_turn(previous_turn, 135);
    }
```

```
    else if(QTI_right_curr_state == Black){ //if right side his border,
turn left
      Hit_Border_Right_QTI();
      previous_turn = LEFT_TURN;
    }
    else if(QTI_left_curr_state == Black){ //if left side his border, turn
right
      Hit_Border_Left_QTI();
      previous_turn = RIGHT_TURN;
    }
    else{ //if not hitting border, move forward
      drive_forward();
    }

  }


}

//functions where movement is based on input time
void drive_backward_time(float duration){
 PORTD = 0b01100000;
 for(int i = 0; i < (int)duration; i++){
  _delay_ms(1);
 }
 stop_moving();
}
void drive_forward_time(float duration){
 PORTD = 0b10010000;
 for(int i = 0; i < (int)duration; i++){
  _delay_ms(1);
 }
 stop_moving();
}
void turn_right_time(float duration){
 PORTD = 0b10100000;
 for(int i = 0; i < (int)duration; i++){
  _delay_ms(1);
 }
 stop_moving();
```

```
}
void turn_left_time(float duration){
 PORTD = 0b01010000;
 for(int i = 0; i < (int)duration; i++){
  _delay_ms(1);
 }
 stop_moving();
}
void stop_moving_time(float duration){
 PORTD = 0b00000000;
 for(int i = 0; i < (int)duration; i++){
  _delay_ms(1);
 }
}

//functions where movement is based on input distance to travel
void drive_backward_distance(float distance){
 distance_time = distance/speed;
 distance_time_ms = distance_time*1000.0;

 distance_time_ms = distance_time_ms*test_distance_multiplier;

 PORTD = 0b01100000;

 //issue where could not do _delay_ms(distance_time_ms), loop fixes issue
 for(int i = 0; i < (int)distance_time_ms; i++){
  _delay_ms(1);
 }

 stop_moving();
}
void drive_forward_distance(float distance){
 distance_time = distance/speed;
 distance_time_ms = distance_time*1000.0;

 distance_time_ms = distance_time_ms*test_distance_multiplier;

 PORTD = 0b10010000;

 for(int i = 0; i < (int)distance_time_ms; i++){
```

```
  _delay_ms(1);
 }

 stop_moving();
}

void turn_right_angle(float angle){ //need to check: turn angle (in
degrees) --> duration motors are on
 arc_distance =
angle*(inches_to_feet(wheel_to_wheel_distance)/2.0)*(pi/180.0);
 angle_time = arc_distance/speed;
 angle_time_ms = angle_time*1000.0;

 angle_time_ms = angle_time_ms*test_angle_multiplier;

 PORTD = 0b10100000;

 for(int i = 0; i < (int)angle_time_ms; i++){
  _delay_ms(1);
 }

 stop_moving();
}
void turn_left_angle(float angle){
 arc_distance =
angle*(inches_to_feet(wheel_to_wheel_distance)/2.0)*(pi/180.0);
 angle_time = arc_distance/speed;
 angle_time_ms = angle_time*1000.0;

 angle_time_ms = angle_time_ms*test_angle_multiplier;


 PORTD = 0b01010000;

 for(int i = 0; i < (int)angle_time_ms; i++){
  _delay_ms(1);
 }

 stop_moving();
}
```

```
//functions for indefinite movement with no input
void drive_backward(){
 PORTD = 0b01100000;
}
void drive_forward(){
 PORTD = 0b10010000;
}
void turn_right(){
 PORTD = 0b10100000;
}
void turn_left(){
 PORTD = 0b01010000;
}
void stop_moving(){
 PORTD = 0b00000000;
}


//functions for basic unit conversions, if needed
float meters_to_feet(float meters){
  feet = meters/3.28;
  return feet;
}

float feet_to_meters(float feet){
  meters = feet*3.28;
  return meters;
}

float inches_to_feet(float inches){
  feet = inches/12.0;
  return feet;
}
```